Objective(s):

- Implement Dijkstra's shortest-path algorithm.

Dijkstra with a PriorityQueue

The PriorityQueue stores pairs in the form {distance, vertex} and always removes the pair with the smallest distance first. When vertex u is processed, examine every edge leaving u. If travelling through u gives a shorter route to v, update distance[v] and prev[v], then add the new pair to the PriorityQueue.

Term	Meaning in this lab
weight	The non-negative cost of travelling along an edge.
INF	There is no edge between the two vertices. The code uses Integer.MAX_VALUE.
distance[v]	The best distance found so far from source to vertex v.
prev[v]	The vertex immediately before v on the current shortest path. Use -1 when none exists.
relax an edge	Test source -> u -> v. Update v when the new route is shorter than its current distance.

```
distance[:] = INF;  prev[:] = -1;  distance[source] = 0
pq.add({0, source})
while pq is not empty:
    {dist, u} = pq.poll()
    skip u if this pair is older than the best known distance
    for each edge u -> v with weight w:
        newDist = distance[u] + w
        if newDist < distance[v]: //relax an edge
            distance[v] = newDist
            prev[v] = u
            pq.add({newDist, v})
```

Vertex states during Dijkstra's algorithm

While Dijkstra's algorithm runs, a vertex can be understood in three states:

1. unvisited / undiscovered — no path to the vertex is known yet, so its distance is INF.

2. discovered — a path to the vertex has been found, so distance[v] has a finite value. The PriorityQueue may contain one or more entries for this vertex because a later relaxation may find a shorter path.

3. explored / finalized — the vertex is removed from the PriorityQueue with its best distance, and its outgoing edges are relaxed. With non-negative edge weights, no later path can improve its distance.

Handling stale PriorityQueue entries

At first, every vertex is unvisited / undiscovered, and its distance is INF. When relaxation finds a path to vertex v, v becomes discovered: distance[v] is updated and a {distance, v} pair is added to the PriorityQueue. If a later relaxation finds a shorter path, Java's PriorityQueue cannot update the older pair. The program adds a new, better pair instead, so both pairs remain in the queue. When the best pair for v is removed, v becomes explored / finalized and its outgoing edges are relaxed. Any older pair for the same finalized vertex is a stale entry and must be skipped. The easiest way to handle stale entries is to use a boolean[] visited array.

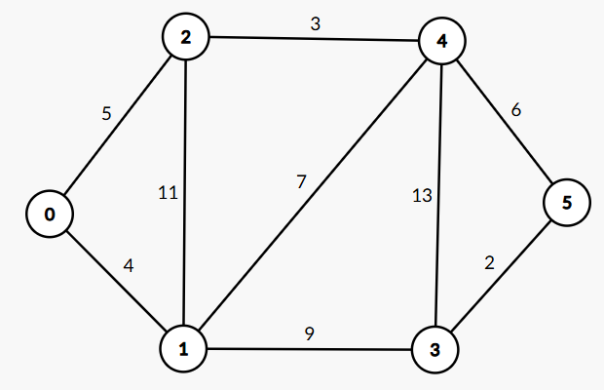
Task 1: Dijkstra on an adjacency matrix

Complete Task 1 in Lab12b_Graph2_XXYYYY.java. Implement Dijkstra's algorithm on the adjacency matrix and reconstruct the path from the prev array.

Method q4() passes a 6 x 6 adjacency matrix to dijkstraWithVisitedArray(graph, 0). Vertices A through F are numbered 0 through 5. A value of 0 on the diagonal represents the same vertex; INF means that no edge exists. All edge weights are non-negative. The matrix represents an undirected graph.

```
        for (int v = 0; v < n; v++) {
            if (graph[u][v] != 0 && graph[u]
                /* your code 1: calculate th
                int newDist = 0;
                /* your code 2: compare the
                if (false) {
                    /* your code 3: update c
pq */
                }
            }
        }
        System.out.println("exploring " + u + " " + Arrays.toString(distance));
    }

    System.out.println("prev= " + Arrays.toString(prev));
    q4ExtractPath(prev, 4);                          // A -> C -> E
    // exploring 0 [0, 4, 5, 2147483647, 2147483647, 2147483647]
    // exploring 1 [0, 4, 5, 13, 11, 2147483647]
    // exploring 2 [0, 4, 5, 13, 8, 2147483647]
    // exploring 4 [0, 4, 5, 13, 8, 14]
    // exploring 3 [0, 4, 5, 13, 8, 14]
    // exploring 5 [0, 4, 5, 13, 8, 14]
    // prev= [-1, 0, 0, 1, 2, 4]
    // path= 0 -> 2 -> 4
}
```



Dijkstra with a Visited Array

This version uses boolean[] visited to record whether vertex u has already been finalized with its shortest distance. When u is first removed from the PriorityQueue, mark it as visited and relax its outgoing edges. If another, older entry for u is removed later, visited[u] is already true, so skip that entry immediately.

An alternative technique is:

Dijkstra with a Stale-Entry Check

This version does not use a visited[] array. Instead, it compares the distance stored in the pair removed from the PriorityQueue with the current best distance in distance[u]. If minCostToU is greater, the pair is an outdated stale entry and is skipped. With non-negative edge weights, this produces the same shortest-path result as the visited-array version.

Task 2: Dijkstra on a weighted adjacency list

Complete Task 2 in Lab12b_Graph2_XXYYYY.java. Implement Dijkstra's algorithm on the weighted adjacency list and return {furthestCity, maxDistance}.

Given an edge list. Each input edge is {from, to, weight}; for example, {0, 1, 4} connects vertices 0 and 1 with weight 4. Every graph.get(u) item is {weight, destination}. All edge weights are non-negative. The graph is undirected, so each input edge is stored in both directions.

Early exit: After a non-stale pair for a vertex is removed from the PriorityQueue, that vertex has been finalized with its shortest distance. Count each finalized vertex. For your code 10, after incrementing settledCount, use break when **settledCount == numVertices**. At that point, all remaining PriorityQueue entries, if any, are stale, so the algorithm need not examine the final vertex's outgoing edges. If some vertices are unreachable from the start, the count never reaches the total and the loop ends normally when the PriorityQueue becomes empty.

```
    System.out.println("Dijkstra (adjacency list) from C:");
    dijkstraWithStaleEntryCheck(graph, 2);
    // exploring 2 [2147483647, 2147483647, 0, 2147483647, 2147483647, 2147483647]
    // exploring 4 [5, 11, 0, 2147483647, 3, 2147483647]
    // exploring 0 [5, 10, 0, 16, 3, 9]
    // exploring 5 [5, 9, 0, 16, 3, 9]
    // exploring 1 [5, 9, 0, 11, 3, 9]
    // exploring 3 [5, 9, 0, 11, 3, 9]
    // prev= [2, 0, -1, 5, 2, 4]
    // Furthest city is 3, distance= 11
}
```

Submission: Lab12b_Graph2_XXYYYY.java where. XX are the first two digit of your student id and YYYY are the last four.

Due date: TBA